BusyLean: Naming Ever Larger Numbers in Lean

Research notes on constructive large numbers with formal comparisons

BusyLean contributors

October 2026

Abstract

We port Cody Roux's *name the biggest number* contest from Coq to Lean 4. In the contest every number must be a closed, computable definition, and every claim "my number is bigger" must be a kernel-checked theorem. We give an exact port of the incumbent Contender 5 (the largest numeral of a raw System T term of depth 42). We review and rebuild five submitted proposals, and we construct Contender 6, which combines their best ideas: an object language with W-types over every finite type, relativized to oracle families, and reflected along every tree that the language itself can define. Along the way we prove formally that one proposal's re-implementation of the old language agrees with the exact port. The notes then explain the strategy for the next contenders, which climb the ladder System F, System F_ω, Calculus of Constructions. Each rung is a qualitatively stronger total language whose normalization can be justified inside Lean.

Contents

1 The game **2**

2 How to compare unimaginably large numbers **2**

3 Contender 5, exactly **2**

4 Five proposals **3**

5 Contender 6 **3**

5.1 The object language . . . 3
5.2 Oracles for free . . . 3
5.3 Reflection along the language's own trees . . . 4
5.4 What is proved . . . 4

6 Engineering lessons **4**

7 Roadmap: impredicativity **5**

8 Status **5**

1 The game

Scott Aaronson's *Who can name the bigger number?* asks for the largest number one can describe concisely. Cody Roux's repository name-the-biggest-number adds two caveats that turn the game into one about *proof*. First, numbers must be *constructive*: computable in principle, with no Busy Beavers. Second, each new contender must come with a *formal proof* that it beats the previous one. The original contest uses Coq. Its last entry, Contender 5 by Samuel Gruetter, is the largest value of a raw STLC+NatRec term (Gödel's System T) of syntax depth at most 42.

The Lean edition. This repository keeps the contest but switches to Lean 4 (version 4.34.0). The rules (`README.md`) are: closed computable `Nat` definitions; a theorem `contender_(N-1)_lt_contender_N`; the axioms `propext` and `Quot.sound` allowed and `Classical.choice` forbidden; at most 15 s to check the definition and 60 s to check the proof, from a clean build; and no laziness or shenanigans. The axiom policy is deliberate. `Classical.choice` is the only one of Lean's three axioms that conflicts with computability, so it is the one that must go.

2 How to compare unimaginably large numbers

All numbers here are far too large to evaluate. The formal theorems compare them directly, but informal *size* is best discussed with the *fast-growing hierarchy* f_α indexed by ordinals: $f_0(n) = n+1$, $f_{\alpha+1}(n) = f_\alpha^n(n)$, and $f_\lambda(n) = f_{\lambda[n]}(n)$ at limits. A total language L whose programs are all provably terminating in a theory T has its bounded-size maxima growing roughly like $f_{|T|}$, where $|T|$ is the proof-theoretic ordinal of T. Two heuristics organize everything below.

1. **The base language dominates.** System T is calibrated by ε_0. T plus one type of countably branching trees, with recursion into all finite types, is calibrated by the Bachmann–Howard ordinal $\mathrm{BH} = \psi(\varepsilon_{\Omega+1})$. With tree classes at every finite level the calibration is $\psi(\Omega_\omega)$, the ordinal of Π^1_1-CA_0.

2. **Reflection multiplies.** Relativizing a language L to an oracle that grows like f_β yields growth like $f_{\beta+|L|}$. Iterating this along an index of height γ yields $f_{|L|\cdot\gamma}$. Explicit ordinal notations without collapsing functions stay below BH, so a stack of reflections over a weak base cannot catch up with a stronger base.

These are informal calibrations, not theorems; the formal content of every contender is a strict inequality checked by Lean's kernel.

3 Contender 5, exactly

`BusyLean/Contender5.lean` ports `Contender.v` one definition at a time. The port keeps every quirk that can affect the value: raw ill-typed terms are interpreted with `error` defaults; variables use reversed de Bruijn indices; lambdas carry both annotations and cast their body; arrow casts recurse only when `type_eqb` succeeds; the enumeration order is the original one, including right-nested `++`; and the winner is chosen by `maxBy`'s strict update, starting from `t0`. A smoke test reproduces the reference table: $0, 2, 10, 146, 24976$ terms and maxima $0, 0, 1, 2, 3$ at depths 0–4.

Agreement of an independent port. Proposal P5 re-implemented the old language *inside* its larger W-language, with W-aware casts and a plain maximum instead of `maxBy`, and asserted without proof that the result is Contender 5. We prove it:

Theorem 1 (`PortAgreement.largest_eq_oldMaximum`)**.** *For every depth n,*

$$\mathit{C5.largestSTLCNatRecNatOfDepth}\ n\ =\ \mathit{BigNumber.oldMaximum}\ n.$$

The proof is a logical relation between the two interpreters: equality at $\mathbb{N}$, preservation at arrows. That avoids function extensionality. Casts in both directions preserve the relation, including the error branches. The fundamental lemma covers reversed lookup, binder casts and natural recursion. P5's enumeration is the translated image of the original, in the same order, and the `maxBy` winner attains the maximum.

4 Five proposals

Five proposals arrived as archives; `proposals/REVIEW.md` has the details. All were rebuilt on Lean 4.34.0, and the two that had never been compiled (P4, P5) needed small repairs, recorded in separate commits.

	base language	amplifier	informal size
P1	System T	FGH over explicit Veblen-style trees	$f_{\varepsilon_0+\alpha}$, $\alpha < \mathrm{BH}$
P2	T + Brouwer trees + oracles	forest reflection on explicit trees	$f_{\mathrm{BH}\cdot\alpha}$
P3	T + unary oracle	reflection on explicit trees	$f_{\varepsilon_0\cdot\alpha}$
P4	T + Brouwer trees + oracle	ramified reflection	$f_{\mathrm{BH}\cdot\alpha}$
P5	T + W-types at all finite types	two diagonals	$f_{\psi(\Omega_\omega)+\omega^2}$

P5 has by far the strongest base and the weakest amplifier, which suggests combining it with the others' amplifiers.

5 Contender 6

5.1 The object language

The language is P5's. Types are generated by $A, B ::= \mathbb{N} \mid A \to B \mid W\,A$, where $W\,A$ is the type of well-founded trees branching over A. Terms are raw (ill-typed terms default), with natural recursion and a W-recursor into every type. Because A ranges over all types, the language has tree classes of every finite level, $W\mathbb{N}$, $W(W\mathbb{N})$, …, and its calibration is $|L| \approx \psi(\Omega_\omega)$.

5.2 Oracles for free

P5's interpreter resolves variables by oldest-first levels. Evaluating a program in the one-element environment $[\langle \mathbb{N} \to \mathbb{N} \to \mathbb{N}, fs\rangle]$ therefore makes `var 0` denote the oracle family fs everywhere, even under binders. No new syntax is needed, and the smoke test confirms it: `var0 3 4` evaluates to $fs(3,4)$, and so does `(fun y => var0 y y) 5` with the family applied at $(5,5)$.

5.3 Reflection along the language's own trees

```
def reflect (fs : Family) (n : ℕ) : ℕ :=
  n + 1 + max (prefixMax (fun i => fs i n) n)
              (maximum (evalWith fs) (allTerms n))
def stage (seed : Family) : WTree ℕ → ℕ → ℕ
  | .leaf   => reflect seed
  | .node f => reflect (fun i => stage seed (f i))
def treesOfDepth (k : ℕ) : WTree ℕ :=
  .node fun i => defTree (nthD (allTerms k) i .zero)
def index : WTree ℕ := .node treesOfDepth
def contender_6 : ℕ := stage p5Family index 42
```

At a node the oracle family *is* the family of child stages, so a program may call any child at a computed index (P2's forest reflection). The new ingredient is `index`: its k-th child joins every $W\mathbb{N}$ tree denoted by a program of depth $\leq k$. The hierarchy therefore runs along the language's own ordinals, all of $|L|$, instead of an explicit notation system below BH. By the multiplication heuristic, Contender 6 is about $f_{|L|\cdot|L|}(42)$.

5.4 What is proved

- `contender_5_lt_contender_6`: the exact port of Contender 5 is below Contender 6.
- `p5_lt_contender_6` and `p5Family_lt_finalFamily`: P5 is beaten at every input.
- `defined_tree_eventually_lt`: every tree the language defines is eventually dominated.
- `program_lt_finalFamily`: every program of depth $\leq n$, run with the final oracles, is exceeded.

All of them depend only on `propext`. The definitions check in about 5 s and the proofs in about 6 s. A machine-checked audit shows that the definition never mentions `contender_5` or its winner selection. It also reports, candidly, that P5's seed contains the old language's maximum as a function of depth.

6 Engineering lessons

- **Toolchain drift changes axiom lists.** In Lean 4.34 a wildcard match arm compiles to a `_sparseCasesOn` helper built on `Nat.land`. `Nat.bitwise`'s termination proof uses `propext`, so "axiom-free on 4.19" did not survive the upgrade. The same happens to core `xs[i]?`. Exhaustive matches avoid it.
- **Tactics carry axioms.** `omega`'s certificates use `funext`, and so `Quot.sound`. Two calls were replaced by explicit lemma chains.
- **Never let the unifier normalize a contender.** Writing `iterate f 41 (f 42)` against `iterate f 42 42` made Lean try to evaluate the number. Generalizing the statement first, so that only $42 = 41 + 1$ is checked, fixes it.
- **Prove inequalities by inclusion, not monotonicity.** Reflection is not monotone in its oracle, because a program may test oracle values. Every comparison therefore exhibits an explicit occurrence: a visible oracle index or an enumerated program.

7 Roadmap: impredicativity

Contender 6 lives below the next collapsing level: everything it does is predicative. The next qualitative jump is *impredicative polymorphism*.

System F. The numeric functions definable in Girard's System F on Church numerals are exactly those provably total in second-order arithmetic Z_2. $\psi(\Omega_\omega)$ is the ordinal of Π^1_1-CA$_0$, a small fragment of Z_2. A System-F maximum therefore dwarfs everything above.

Why Lean can host it. System F has no set-theoretic model in Lean's predicative `Type`. Girard's *reducibility candidates* proof of strong normalization quantifies only over *predicates on terms*, and Lean's impredicative `Prop` makes $\forall C : \mathrm{Term} \to \mathrm{Prop}$ a legitimate proposition. The plan:

1. untyped de Bruijn λ-terms with β-reduction (types erased);
2. Curry-style System F typing; saturated sets; the interpretation $[\![\forall X.A]\!]_\rho = \bigcap_C [\![A]\!]_{C\cdot\rho}$; and adequacy, which gives strong normalization;
3. Church-style terms (decidable type checking) enumerated by depth, checked against the Church numeral type, erased, normalized by a deterministic strategy (a total function by well-founded recursion, using the SN proof), and decoded;
4. an *oracle constant* O with δ-rule $O\,\overline{n} \to \overline{f(n)}$, so System F programs can amplify a previous contender's family. That makes the formal comparison immediate, and it lets F's own strength act on top of it.

Further rungs. System F$_\omega$ (higher-order arithmetic) needs candidates for type constructors, i.e. functions between candidate spaces. The Calculus of Constructions, the system behind Loader's number, adds dependent types. Each rung is a new contender.

8 Status

Contender	idea	informal size
5 (Coq)	System T, depth 42	$f_{<\varepsilon_0}(42)$
6 (Lean)	forest reflection over finite-type W-language, own trees	$f_{\psi(\Omega_\omega)^2}(42)$
7 (planned)	System F with oracle	beyond $f_{\lvert\mathsf{Z}_2\rvert}$